Offer Server API v2 documentation

Owner	Ⓢ Stjepan Matijasevic
Category	Tech spec
Tags	3rd parties API Offer
Status	Done
Archive	☐

Goal
Base URL
- Stage environment
- Production environment
- List of possible locale values for `:lang` path param

REST endpoints
Streaming-api - v2 endpoints

Goal

To have clean offer.api.v2 specification on one place, containing all necessary information for client teams integration.

V2 REST endpoints contains multi-language support - we can specify locale for example nl-BE, or ro-RO to get different translation in local<field_name> map.

Also, some targets (like Napoleon) can have mandatory translation. So for Napoleon if you ask translation in fr-BE (French) you will get in French plus in English (en-BE)

Base URL

Stage environment

On stage environment we serve REST and SSE endpoints via Fastly

URLs:

- **Romania**: https://stage-superbet-offer-ro.freetls.fastly.net/
- **Poland**:https://stage-superbet-offer-pl.freetls.fastly.net/
- **Serbia**: https://offer-server.trading.stage.superbet.com/sb-rs/api/
- **Napoleon**: https://stage-superbet-offer-ng-be.freetls.fastly.net/
- **Brazil**: https://stage-superbet-offer-basic.freetls.fastly.net/sb-basic/api/

Production environment

On production environment we serve REST and SSE endpoints via Fastly

URLs:

- **Romania**: https://production-superbet-offer-ro.freetls.fastly.net/
- **Poland**: https://production-superbet-offer-pl.freetls.fastly.net/
- **Serbia**: https://production-superbet-offer-basic.freetls.fastly.net/sb-rs/api
- **Napoleon**: https://production-superbet-offer-ng-be.freetls.fastly.net/
- **Brazil**: https://production-superbet-offer-basic.freetls.fastly.net/sb-basic/api

▼ List of possible locale values for `:lang` path param

- **nl-BE** (Dutch - Belgium)
- **en-BE** (English - Belgium)
- **fr-BE** (French - Belgium)
- **sr-Latn-RS** (Serbian - Serbia in latin)
- **en-GB** (English - United Kingdom)
- **pl-PL** (Polish - Poland)

- **ro-RO** (Romanian - Romania)
- **hr-HR** (Croatian - Croatia)
- **pt-BR** (Portuguese - Brazil)
- **en-BR** (Englis - Brazil)

▼ REST endpoints

For all REST response, response model is in json format in form:

```
{
    "error": false,
    "dataIn": {
        "lang": "en-GB"
    },
    "data": {<specific-data-model-for-endpoint>}
```

Where:

- **error** - indicate error in fetching data
- **dataIn** - represent path and query params for REST endpoint
- **data** - requested data in `lang` locale - specific model for almost every endpoint. It will be defined for every endpoint in rest of document

Structure data

▼ Get offer structure by locale

Path: `/v2/:lang/struct`

Used to get offer structure name and order for sport, category, tournaments and outcomes plus marketTree

Path params:

- `lang` - locale for desired translation

Query params:

- no query parameters

Response:

Model of data field in response is:

```
// Struct structure
type Struct struct {
    Sports      []*Sport      `json:"sports"`
    Categories  []*Category   `json:"categories"`
    Tournaments []*Tournament `json:"tournaments"`
    Outcomes    []*Outcome    `json:"outcomes"`   //old `js
    MarketTree  []*SportNode  `json:"marketTree"` //old `js
}
```

For more information whole model is at the bottom of this document under **"Complete data model"** section.

Example of response:

v2-lang-struct-response.json

▼ Get market-groups by locale, sport and phase

Path: `/v2/:lang/sport/:sportId/phase/:phase/market-groups`

Path params:

- `lang` - locale for desired translation
- `sportId` - show market-groups for sportId
- `phase` - show market-groups for phase (inplay/prematch)

Query params:

- no query parameters

Response:

Model of data field in response is:

```
// MarketGroup structure
type MarketGroup struct {
    ID         int64             `json:"id"`
    Order      int32             `json:"order"`
    SportID    int64             `json:"sportId"`
    LocalNames map[string]string `json:"localNames"`
    Markets    []int64           `json:"markets"`
}
```

For more information whole model is at the bottom of this document under **"Complete data model"** section.

Example of response:

v2-lang-sport-phase-market-groups.json

▼ Get tournaments by locale and sportId

Path: `/v2/:lang/sport/:sportId/tournaments`

Path params:

- `lang` - locale for desired translation
- `sportId` - show tournaments for sportId

Query params:

- no query parameters

Response:

Model of data field in response is:

```
// CategoryWithTournaments structure
type CategoryWithTournaments struct {
    SportID      int64                    `json:"sportId"`
    CategoryID   int64                    `json:"categoryId"
    LocalNames   map[string]string        `json:"localNames"
    Competitions []*SportsTreeTournament  `json:"competitio
    Order        int32                    `json:"-"`
    Name         string                   `json:"-"`
}
```

For more information whole model is at the bottom of this document under **"Complete data model"** section.

Example of response:

v2-lang-sport-tournaments.json

▼ Event data

▼ Get Event by `eventId` in `lang` locale

Path: `/v2/:lang/events/:eventId`

Path params:

- `lang` - locale for desired translation
- `eventId` - eventId to fetch

Query params:

- `oddsResults` - boolean, not mandatory - if true - we return whole event with results (oddsResults)

Response:

Model of data field in response is:

```
// Event structure
type Event struct {
    UUID                        string                `j
    MatchID                     int64                 `j
    MatchDate                   string                `j
    HomeTeamID                  string                `j
    AwayTeamID                  string                `j
    MatchCode                   int64                 `j
    IncrementID                 int64                 `j
    OfferID                     int64                 `j
    SportID                     int64                 `j
    CategoryID                  int64                 `j
    TournamentID                int64                 `j
    SpecialOfferID              string                `j
    Metadata                    *Metadata             `j
    BetRadarID                  string                `j
    MatchTags                   string                `j
    MatchFooter                 string                `j
    MatchPromotedLiveOrder      int32                 `j
    TournamentPromotedLiveOrder int32                 `j
    MatchTimestamp              int64                 `j
    HasLive                     bool                  `j
    MatchName                   string                `j
    SpecialOfferIDs             []string              `j
    OfferStateStatus            map[OfferState]string `j
    HasStream                   map[int32]string      `j
    BetGroupCount               int32                 `j
```

```
    Counts                      Counts                `j
    Odds                        []Odd                 `j
    OddsResults                 []Odd                 `j
    MatchResults                []MatchResult         `j
    ResolvingRules              []Rule                `j
    UtcDate                     string                `j
    EventCategory               string                `j
    Streams                     []string              `j
}
```

For more information whole model is at the bottom of this document under **“Complete data model”** section.

Example of response:

v-lang-events-eventId-preselected.json

▼ Get event by `lang` locale and `betradarId`

Path: `/v2/:lang/events/betradar/:betradarId`

Path params:

- `lang` - locale for desired translation
- `betradarId` - Betradar id of event to fetch

Query params:

- `oddsResults` - boolean, not mandatory - if true - we return whole event with results (oddsResults)

Response:

Same response as for previous endpoint, get event by eventId

```
// Event structure
type Event struct {
    UUID                        string                `j
    MatchID                     int64                 `j
    MatchDate                   string                `j
    HomeTeamID                  string                `j
    AwayTeamID                  string                `j
    MatchCode                   int64                 `j
    IncrementID                 int64                 `j
    OfferID                     int64                 `j
    SportID                     int64                 `j
    CategoryID                  int64                 `j
    TournamentID                int64                 `j
    SpecialOfferID              string                `j
    Metadata                    *Metadata             `j
    BetRadarID                  string                `j
    MatchTags                   string                `j
    MatchFooter                 string                `j
    MatchPromotedLiveOrder      int32                 `j
    TournamentPromotedLiveOrder int32                 `j
    MatchTimestamp              int64                 `j
    HasLive                     bool                  `j
    MatchName                   string                `j
    SpecialOfferIDs             []string              `j
    OfferStateStatus            map[OfferState]string `j
    HasStream                   map[int32]string      `j
    BetGroupCount               int32                 `j
    Counts                      Counts                `j
    Odds                        []Odd                 `j
    OddsResults                 []Odd                 `j
    MatchResults                []MatchResult         `j
    ResolvingRules              []Rule                `j
    UtcDate                     string                `j
    EventCategory               string                `j
```

```
    Streams                        []string                `j
}
```

For more information whole model is at the bottom of this document under **“Complete data model”** section.

Example of response:

v2-lang-events-betradarId-preselected.json

▼ Get array of events by date

Path: `/v2/:lang/events/by-date`

Path params:

- `lang` - locale for desired translation

Query params:

- `startDate` - **mandatory**, in iso8601 timeFormat: 2006-01-02 15:04:05 - from which date/time we search eventsf
- `endDate` - not mandatory - in iso8601 timeFormat: 2006-01-02 15:04:05 - till which date/time we search event
- `sportId` - not mandatory- for which sportId we search events
- `categoryId` - not mandatory- for which category we search/filter events
- `tournamentIds` - not mandatory- for which tournaments we search/filter events (array → comma-separated string)
- `currentStatus` - not mandatory- for which offerStatus we search/filter events (can be: **active** or **finished**)

- `offerState` - not mandatory- for which offerState we search/filter events (can be: **prematch**, **live**, **inplay** - inplay == live, we kept both values for historic reason)

Response:

Model of data field in response is: array of Events where Event is defined like in previous event endpoints

```
// Event structure
type Event struct {
    UUID                        string                `j
    MatchID                     int64                 `j
    MatchDate                   string                `j
    HomeTeamID                  string                `j
    AwayTeamID                  string                `j
    MatchCode                   int64                 `j
    IncrementID                 int64                 `j
    OfferID                     int64                 `j
    SportID                     int64                 `j
    CategoryID                  int64                 `j
    TournamentID                int64                 `j
    SpecialOfferID              string                `j
    Metadata                    *Metadata             `j
    BetRadarID                  string                `j
    MatchTags                   string                `j
    MatchFooter                 string                `j
    MatchPromotedLiveOrder      int32                 `j
    TournamentPromotedLiveOrder int32                 `j
    MatchTimestamp              int64                 `j
    HasLive                     bool                  `j
    MatchName                   string                `j
    SpecialOfferIDs             []string              `j
    OfferStateStatus            map[OfferState]string `j
    HasStream                   map[int32]string      `j
```

```
    BetGroupCount                int32                  `j
    Counts                       Counts                 `j
    Odds                         []Odd                  `j
    OddsResults                  []Odd                  `j
    MatchResults                 []MatchResult          `j
    ResolvingRules               []Rule                 `j
    UtcDate                      string                 `j
    EventCategory                string                 `j
    Streams                      []string               `j
}
```

For more information whole model is at the bottom of this document under **“Complete data model”** section.

Example response:

v2-by-date.json

▼ Get topten events

Path: `/v2/:lang/events/topten`

Path params:

- `lang` - locale for desired translation

Query params:

- no query parameters

Response:

Model of data field in response is: array of Events where Event is defined like in previous event endpoints:

```
// Event structure
type Event struct {
    UUID                        string                `j
    MatchID                     int64                 `j
    MatchDate                   string                `j
    HomeTeamID                  string                `j
    AwayTeamID                  string                `j
    MatchCode                   int64                 `j
    IncrementID                 int64                 `j
    OfferID                     int64                 `j
    SportID                     int64                 `j
    CategoryID                  int64                 `j
    TournamentID                int64                 `j
    SpecialOfferID              string                `j
    Metadata                    *Metadata             `j
    BetRadarID                  string                `j
    MatchTags                   string                `j
    MatchFooter                 string                `j
    MatchPromotedLiveOrder      int32                 `j
    TournamentPromotedLiveOrder int32                 `j
    MatchTimestamp              int64                 `j
    HasLive                     bool                  `j
    MatchName                   string                `j
    SpecialOfferIDs             []string              `j
    OfferStateStatus            map[OfferState]string `j
    HasStream                   map[int32]string      `j
    BetGroupCount               int32                 `j
    Counts                      Counts                `j
    Odds                        []Odd                 `j
    OddsResults                 []Odd                 `j
    MatchResults                []MatchResult         `j
    ResolvingRules              []Rule                `j
    UtcDate                     string                `j
    EventCategory               string                `j
```

```
    Streams                         []string                `j
}
```

For more information whole model is at the bottom of this document under **“Complete data model”** section.

Example of response:

v2-topten.json

▼ Get events that meet search string

Path: `/v2/:lang/events/search/:search`

Path params:

- `lang` - locale for desired translation
- `search` - return events that contains this string

Query params:

- `startDate` - **mandatory**, in iso8601 timeFormat: 2006-01-02 15:04:05 - from which date/time we search events

Response:

Model of data field in response is: array of Events where Event is defined like in previous event endpoints:

```
// Event structure
type Event struct {
    UUID                        string                  `j
    MatchID                     int64                   `j
    MatchDate                   string                  `j
    HomeTeamID                  string                  `j
```

```
    AwayTeamID                    string                  `j
    MatchCode                     int64                   `j
    IncrementID                   int64                   `j
    OfferID                       int64                   `j
    SportID                       int64                   `j
    CategoryID                    int64                   `j
    TournamentID                  int64                   `j
    SpecialOfferID                string                  `j
    Metadata                      *Metadata               `j
    BetRadarID                    string                  `j
    MatchTags                     string                  `j
    MatchFooter                   string                  `j
    MatchPromotedLiveOrder        int32                   `j
    TournamentPromotedLiveOrder int32                     `j
    MatchTimestamp                int64                   `j
    HasLive                       bool                    `j
    MatchName                     string                  `j
    SpecialOfferIDs               []string                `j
    OfferStateStatus              map[OfferState]string `j
    HasStream                     map[int32]string        `j
    BetGroupCount                 int32                   `j
    Counts                        Counts                  `j
    Odds                          []Odd                   `j
    OddsResults                   []Odd                   `j
    MatchResults                  []MatchResult           `j
    ResolvingRules                []Rule                  `j
    UtcDate                       string                  `j
    EventCategory                 string                  `j
    Streams                       []string                `j
}
```

For more information whole model is at the bottom of this document under **“Complete data model”** section.

Example of response:

vs-search.json

▼ Get outrights in `lang` locale

Path: `/v2/:lang/events/outrights/all`

Path params:

- `lang` - locale for desired translation

Query params:

- no query parameters

Response:

Model of data field in response is: array of Events where Event is defined like in previous event endpoints:

```
// Event structure
type Event struct {
    UUID                        string                `j
    MatchID                     int64                 `j
    MatchDate                   string                `j
    HomeTeamID                  string                `j
    AwayTeamID                  string                `j
    MatchCode                   int64                 `j
    IncrementID                 int64                 `j
    OfferID                     int64                 `j
    SportID                     int64                 `j
    CategoryID                  int64                 `j
    TournamentID                int64                 `j
    SpecialOfferID              string                `j
    Metadata                    *Metadata             `j
    BetRadarID                  string                `j
    MatchTags                   string                `j
```

```
    MatchFooter                  string                `j
    MatchPromotedLiveOrder       int32                 `j
    TournamentPromotedLiveOrder  int32                 `j
    MatchTimestamp               int64                 `j
    HasLive                      bool                  `j
    MatchName                    string                `j
    SpecialOfferIDs              []string              `j
    OfferStateStatus             map[OfferState]string `j
    HasStream                    map[int32]string      `j
    BetGroupCount                int32                 `j
    Counts                       Counts                `j
    Odds                         []Odd                 `j
    OddsResults                  []Odd                 `j
    MatchResults                 []MatchResult         `j
    ResolvingRules               []Rule                `j
    UtcDate                      string                `j
    EventCategory                string                `j
    Streams                      []string              `j
}
```

For more information whole model is at the bottom of this document under **“Complete data model”** section.

Example of response:

v2-outrights.json

▼ Get promoted events in `lang` in locale

Path: `/v2/:lang/events/promoted`

Path params:

- `lang` - locale for desired translation

Query params:

- no query parameters

Response:

Model of data field in response is:

```
type Promoted struct {
    SuperKvota  []*Event     `json:"superKvota"`
    SuperExtra  []*Event     `json:"superExtra"`
    TopTen      []*Event     `json:"topTen"`
}
```

For more information whole model is at the bottom of this document under **“Complete data model”** section.

Example of response:

v2-promoted.json

▼ Get active offer per dates

🏗️ Currently on dev env

Path: `/v2/active-offer-per-dates`

Path params:

- no path params

Query params:

- **sportID** - if we define this parameter active offer per dates will be returned only for that sport otherwise endpoint will return active offer per dates for all sports

Response:

Model of data field in response is:

```
// EventCountV2 model
type EventCountV2 struct {
    Key   KeyV2 `json:"_id"`
    Value int   `json:"count"`
}

// KeyV2 model
type KeyV2 struct {
    Date    string `json:"date"`
    SportID int64  `json:"sportId"`
}
```

Example of response:

active-offer-per-dates.json

▼ Streaming-api - v2 endpoints

Base URL:

All endpoints are same except for Serbia

Stage environment

On stage environment we serve REST and SSE endpoints via Fastly

URLs:

- **Romania**: https://stage-superbet-offer-ro.freetls.fastly.net/
- **Poland**:https://stage-superbet-offer-pl.freetls.fastly.net/
- **Serbia**: https://offer-server.trading.stage.superbet.com/sb-rs/api/

- **Napoleon**: https://stage-superbet-offer-ng-be.freetls.fastly.net/

Production environment

On production environment we serve REST and SSE endpoints via Fastly

URLs:

- **Romania**: https://production-superbet-offer-ro.freetls.fastly.net/
- **Poland**: https://production-superbet-offer-pl.freetls.fastly.net/
- **Serbia**: https://production-superbet-offer-basic.freetls.fastly.net/sb-rs/api/
- **Napoleon**: https://production-superbet-offer-ng-be.freetls.fastly.net/

SSE endpoints

SSE endpoints are used to get `struct` and `event` updates in realtime

▼ Get struct updates

Path: `/subscription/v2/:lang/structure`

This endpoint is used to get `sport` , `category` and `tournaments` updates in realtime.

Path params:

- `lang` - locale for desired translation

Response:

`data` field of each message will have `resourceId`, `timestamp`, `id`, `type` and `data` fields. `type` field indicates the part of the structure that is changed. The type can be one of the following:

- `sport`
- `category`
- `tournament`

`data` field will contain different data depending on the `type` field:

▼ `sport` type

`data` field contains:

- `id` that represent sportId
- `localNames` with key-value pair where the key is requested locale from pathParam and extra locale that represent mandatory translation
- `order` sport order value

```
{
    "resourceId": "sport:2",
    "type": "sport",
    "id": 2,
    "data":
    {
        "id": "2",
        "localNames":
        {
            "en-BE": "Tennis",
            "nl-BE": "Tennis"
        },
        "order": 267
    },
    "timestamp": 1694414172
}
```

▼ `category` type

`data` field contains:

- `id` that represent categoryId
- `localNames` with key-value pair where the key is requested locale from pathParam and extra locale that represent mandatory translation
- `order` category order value

```
{
    "resourceId": "category:45",
    "type": "category",
```

```
    "id": 45,
    "data":
    {
        "id": "45",
        "localNames":
        {
            "en-BE": "England",
            "nl-BE": "England."
        },
        "order": 420
    },
    "timestamp": 1694414066
}
```

▼ `tournament` type

`data` field contains:

- `id` that represent tournamentId
- `localNames` with key-value pair where the key is requested locale from pathParam and extra locale that represent mandatory translation
- `order` tournament order value
- `orderPromoted` promoted tournament order
- `footer` - footer text for tournament from Bookie in requested locale (`:lang` param from path)

```
{
    "resourceId": "tournament:106",
    "type": "tournament",
    "id": 106,
    "data":
    {
        "id": "106",
        "localNames":
        {
```

```
            "en-BE": "England - Premier League",
            "nl-BE": "England - Premier League "
        },
        "order": 1541,
        "orderPromoted": 0,
        "footer": "test footer for NL."
    },
    "timestamp": 1694415691
}
```

Example of response:

v2-sse-struct-response.json

▼ Event SSE endpoint

All SSE endpoints are used to get live updates for one or multiple events and response for each endpoint will be same.

Response:

`data` field of each message will have `resourceId`, `event, timestamp` and `data` fields.

`data` field will contain event data in format:

```
// Event structure
type Event struct {
    UUID                        string                `jso
    MatchID                     int64                 `jso
    MatchDate                   string                `jso
    HomeTeamID                  string                `jso
    AwayTeamID                  string                `jso
    MatchCode                   int64                 `jso
    IncrementID                 int64                 `jso
```

```
    OfferID                      int64                  `jso
    SportID                      int64                  `jso
    CategoryID                   int64                  `jso
    TournamentID                 int64                  `jso
    SpecialOfferID               string                 `jso
    Metadata                     *Metadata              `jso
    BetRadarID                   string                 `jso
    MatchTags                    string                 `jso
    MatchFooter                  string                 `jso
    MatchPromotedLiveOrder       int32                  `jso
    TournamentPromotedLiveOrder  int32                  `jso
    MatchTimestamp               int64                  `jso
    HasLive                      bool                   `jso
    MatchName                    string                 `jso
    SpecialOfferIDs              []string               `jso
    OfferStateStatus             map[OfferState]string  `jso
    HasStream                    map[int32]string       `jso
    BetGroupCount                int32                  `jso
    Counts                       Counts                 `jso
    Odds                         []Odd                  `jso
    OddsResults                  []Odd                  `jso
    MatchResults                 []MatchResult          `jso
    ResolvingRules               []Rule                 `jso
    UtcDate                      string                 `jso
    EventCategory                string                 `jso
    Streams                      []string               `jso
}
```

For more information whole model is at the bottom of this document under **“Complete data model”** section.

Example of response:

v2-sse-eventId-reponse.json

Path and description of each endpoint will be describe in rest of document.

▼ Get single event updates

Path: `/subscription/v2/:lang/events/:eventID`

Used to get live updates for one event with `eventID`

Path params:

- `lang` - locale for desired translation
- `eventID` - id of event for which we want to receive updated

▼ Get live updates for all events

Path: `/subscription/v2/:lang/events/all`

Used to get update for all event

Path params:

- `lang` - locale for desired translation

▼ Get live updates for all live events

Path: `/subscription/v2/:lang/events/live`

Used to get all live events

Path params:

- `lang` - locale for desired translation

▼ Get live updates for all prematch events

Path: `/subscription/v2/:lang/events/prematch`

Used to get all prematch events

Path params:

- `lang` - locale for desired translation

▼ Get live updates for all prematch events in tournament

Path: `/subscription/v2/:lang/events/prematch/tournament`

Used to get all prematch events for one or more tournaments

Path params:

- `lang` - locale for desired translation

Query params:

- `ids` - **mandatory**, one or more tournament ids separated with `,`

▼ Get live updates for prematch events in specific sport

Path: `/subscription/v2/:lang/events/prematch/sport/:sportID`

Used to get live updates for prematch events in specific sport

Path params:

- `lang` - locale for desired translation
- `sportID` - sport id

▼ Get live updates for outright events

Path: `/subscription/v2/:lang/events/outrights`

Used to get live updates for **Outright** events

Path params:

- `lang` - locale for desired translation

▼ Complete data model

Here is complete data model

```
// ViewModels struct
type ViewModels struct {
    FullOffer            Event
    FullOfferWithResults Event
    PreselectedOffer     Event
    SuperOffer           Event
}

// Event structure
type Event struct {
    UUID                        string                `j
    MatchID                     int64                 `j
    MatchDate                   string                `j
    HomeTeamID                  string                `j
    AwayTeamID                  string                `j
    MatchCode                   int64                 `j
    IncrementID                 int64                 `j
    OfferID                     int64                 `j
    SportID                     int64                 `j
    CategoryID                  int64                 `j
    TournamentID                int64                 `j
    SpecialOfferID              string                `j
    Metadata                    *Metadata             `j
    BetRadarID                  string                `j
    MatchTags                   string                `j
    MatchFooter                 string                `j
    MatchPromotedLiveOrder      int32                 `j
    TournamentPromotedLiveOrder int32                 `j
    MatchTimestamp              int64                 `j
    HasLive                     bool                  `j
    MatchName                   string                `j
    SpecialOfferIDs             []string              `j
    OfferStateStatus            map[OfferState]string `j
    HasStream                   map[int32]string      `j
    BetGroupCount               int32                 `j
```

```
    Counts                      Counts                `j
    Odds                        []Odd                 `j
    OddsResults                 []Odd                 `j
    MatchResults                []MatchResult         `j
    ResolvingRules              []Rule                `j
    UtcDate                     string                `j
    EventCategory               string                `j
    Streams                     []string              `j
}

// Odd structure
type Odd struct {
    UUID                string            `json:"uuid"`
    MarketUUID          string            `json:"marketU
    BetGroupID          int64            `json:"marketI
    OutcomeID           int64            `json:"outcome
    Price               float64          `json:"price"`
    SpecialBetValue     string            `json:"special
    Status              string            `json:"status"
    Specifiers          map[string]string `json:"specifi
    BetGroupOrder       int32            `json:"marketG
    BetGroupTags        string            `json:"marketT
    BetGroupCode        string            `json:"marketC
    OfferStateID        int32            `json:"offerSt
    ShowSpecialBetValue string            `json:"showSpe
    Tags                string            `json:"tags,om
    Code                string            `json:"code,om
    Extra               map[string]string `json:"extra,o
    Name                string            `json:"name,om
    MarketName          string            `json:"marketN
    Info                string            `json:"info"`
}

// Metadata structure
type Metadata struct {
    HomeTeamScore             string   `json:"homeTeamSc
```

```
    AwayTeamScore              string   `json:"awayTeamSc
    HomeTeamGameScore          string   `json:"homeTeamGa
    AwayTeamGameScore          string   `json:"awayTeamGa
    Periods                    []Period `json:"periods,om
    HomeTeamYellowCards        *int32   `json:"homeTeamYe
    AwayTeamYellowCards        *int32   `json:"awayTeamYe
    HomeTeamRedCards           *int32   `json:"homeTeamRe
    AwayTeamRedCards           *int32   `json:"awayTeamRe
    HomeTeamCorners            *int32   `json:"homeTeamCo
    AwayTeamCorners            *int32   `json:"awayTeamCo
    TeamServing                int32    `json:"teamServin
    Minutes                    string   `json:"minutes,om
    RemainingTime              string   `json:"remainingT
    RemainingTimeInPeriod      string   `json:"remainingP
    StoppageTime               string   `json:"stoppageTi
    StoppageTimeAnnounced      string   `json:"stoppageTi
    Stopped                    *int32   `json:"stopped,om
    Status                     string   `json:"status,omi
    MatchStatusLabelLocalised string   `json:"periodStat
    MatchStatusLabel           string   `json:"matchStatu
    BookingStatus              string   `json:"bookingSta
    BetRadarTimestamp          int64    `json:"brTimestam
    BetRadarID                 int64    `json:"brId,omite
}

// Period struct
type Period struct {
    HomeTeamScore string `json:"homeTeamScore"` //old `j
    AwayTeamScore string `json:"awayTeamScore"` //old `j
    Type          string `json:"type"`          //old `j
    Number        int32  `json:"num"`           //old `j
}

// Counts structure
type Counts struct {
    Odds      map[OfferState]int32 `json:"odds"`
```

```
    BetGroups map[OfferState]int32 `json:"markets"` //ol
}

// MatchResult structure
type MatchResult struct {
    Name  string `json:"name"`  //old  `json:"n"
    Value string `json:"value"` //old  `json:"v"
}

// Struct structure
type Struct struct {
    Sports      []*Sport      `json:"sports"`
    Categories  []*Category   `json:"categories"`
    Tournaments []*Tournament `json:"tournaments"`
    Outcomes    []*Outcome    `json:"outcomes"`   //old
    MarketTree  []*SportNode  `json:"marketTree"` //old
}

// Sport structure
type Sport struct {
    ID         string            `json:"id"`
    LocalNames map[string]string `json:"localNames"`
    Order      int64             `json:"order"`
}

// Category structure
type Category struct {
    ID         string            `json:"id"`
    LocalNames map[string]string `json:"localNames"`
    Order      int64             `json:"order"`
}

// Tournament structure
type Tournament struct {
    ID            string            `json:"id"`
    LocalNames    map[string]string `json:"localNames"`
```

```
    Order         int64             `json:"order"`
    OrderPromoted int64             `json:"orderPromoted
    Footer        string            `json:"footer"`
}

// Outcome structure
type Outcome struct {
    ID         int64  `json:"id"`
    ColumnName string `json:"columnName"` // old `json:"
}

// SportNode structure
type SportNode struct {
    SportID int64         `json:"sportId"` //old `json:"
    Markets []*MarketNode `json:"markets"`
}

// MarketNode structure
type MarketNode struct {
    ID       string  `json:"marketId"` //old `json:"bgi"
    Outcomes []int64 `json:"outcomes"` //old `json:"odds
}

// MarketGroup structure
type MarketGroup struct {
    ID         int64             `json:"id"`
    Order      int32             `json:"order"`
    SportID    int64             `json:"sportId"`
    LocalNames map[string]string `json:"localNames"`
    Markets    []int64           `json:"markets"`
}

// MarketGroups structure
type MarketGroups []*MarketGroup

// SportsTreeTournament structure
```

```
type SportsTreeTournament struct {
    TournamentID int64             `json:"tournamentId"`
    LocalNames   map[string]string `json:"localNames"`
    Order        int32             `json:"-"`
    MasterOrder  int32             `json:"-"`
    Name         string            `json:"-"`
}

// CategoryWithTournaments structure
type CategoryWithTournaments struct {
    SportID      int64                   `json:"sportId"
    CategoryID   int64                   `json:"category
    LocalNames   map[string]string       `json:"localNam
    Competitions []*SportsTreeTournament `json:"competit
    Order        int32                   `json:"-"`
    Name         string                  `json:"-"`
}

// Rule structure
type Rule struct {
    Name     string  `xml:"name,attr" json:"name"`
    Param1   float64 `xml:"param1,attr" json:"param1"`
    Param2   float64 `xml:"param2,attr" json:"param2"`
    Modifier string  `xml:"modifier,attr" json:"modifier
}

// OfferState type
type OfferState int32

// OfferStatus type
type OfferStatus int32

// OfferResult type
type OfferResult int32

// OfferStatePrematch if state is  prematch
```

```go
// OfferStatePrematch if state is  prematch
// OfferStateInplay if state is inplay - live
// OfferStateAny if state is any
const (
    OfferStateAny      = 0
    OfferStatePrematch = 1
    OfferStateInplay   = 2
)

// OfferStatusAny event status is any
// OfferStatusActive event status is active
// OfferStatusFinished event status is finished
const (
    OfferStatusAny OfferStatus = iota
    OfferStatusActive
    OfferStatusFinished
)

// EventCountV2 model
type EventCountV2 struct {
    Key   KeyV2 `json:"_id"`
    Value int   `json:"count"`
}

// KeyV2 model
type KeyV2 struct {
    Date    string `json:"date"`
    SportID int64  `json:"sportId"`
}
```